

Comparative Performance Analysis of Educational Question–Answering using LLM Models

Project & Paper Roadmap for Student Research

September 9, 2026

Document Purpose

This document is a complete, self-contained roadmap for a Computer Science research project and accompanying paper. It is designed so that a student (or small team) can follow it step-by-step, without needing any paid API keys, to compare multiple open-source LLMs on an educational question-answering task using three complementary evaluation metrics.

Contents

1	Project Overview	3
1.1	Title	3
1.2	Problem Statement	3
1.3	Objectives	3
1.4	Why This Project Is Suitable for Students	3
2	System Requirements and Setup (No API)	4
2.1	Hardware Options	4
2.2	Software Stack	4
2.3	Installation Commands	4
2.4	Pulling the Local Models	4
3	Dataset Selection	6
3.1	Recommended Datasets (Free, Hugging Face Hosted)	6
3.2	Recommended Combination	6
3.3	Loading a Dataset	6
4	Models Under Comparison	7
5	Methodology / Pipeline	8
5.1	Pipeline Diagram (Textual)	8
5.2	Fixed Prompt Template	8
6	Evaluation Metrics (The Three Metrics)	9
6.1	Metric 1 – Lexical Overlap: ROUGE-L	9
6.2	Metric 2 – Semantic Similarity: BERTScore	9
6.3	Metric 3 – Correctness / Quality: Local LLM-as-Judge	9
6.4	Metric Summary Table	10

7	Aggregation, Ranking and Statistical Analysis	11
7.1	Aggregation Code	11
7.2	Subject-wise Breakdown	11
7.3	Statistical Significance Testing	11
7.4	Visualization	11
8	Week-by-Week Roadmap	13
9	Optional Extensions (For Stronger Papers / Bonus Marks)	14
10	Recommended Paper Structure	14
11	Deliverables Checklist	14
12	Reference Tools and Documentation Links	15

1 Project Overview

1.1 Title

“EduBench-Local: A Comparative Performance Analysis of Open-Source LLMs on Educational Question Answering Using Multi-Metric Evaluation”

1.2 Problem Statement

Large Language Models (LLMs) are increasingly used as tutoring and study aids, yet little consistent, reproducible evidence exists on *which freely-available, locally-runnable* models perform best for educational question answering across subjects. This project builds a fully offline (no paid API) evaluation pipeline that generates answers from 4–5 open-source LLMs on an educational QA dataset and scores them using three complementary metrics spanning lexical, semantic, and correctness dimensions.

1.3 Objectives

- O1.** Build a reproducible, cost-free pipeline to generate answers from multiple open-source LLMs on the same educational question set.
- O2.** Evaluate model outputs using three metrics: a lexical-overlap metric (ROUGE-L), a semantic-similarity metric (BERTScore), and a correctness/quality metric (local LLM-as-judge or Exact Match/F1).
- O3.** Rank models overall and per-subject, and analyze trade-offs (accuracy vs. verbosity vs. speed vs. hallucination).
- O4.** Produce a research paper documenting methodology, results, and educational implications.

1.4 Why This Project Is Suitable for Students

- No paid API keys or cloud billing required.
- Runs on a normal laptop (CPU) or a free Google Colab / Kaggle GPU session.
- Uses well-documented, actively maintained open-source tools (Ollama, Hugging Face, standard NLP metric libraries).
- Produces both a working software artifact (leaderboard + dashboard) and a publishable-style research paper.

2 System Requirements and Setup (No API)

2.1 Hardware Options

Setup	Requirement	Notes
Local laptop (CPU only)	8 GB+ RAM	Use small models (1–3B parameters); slower generation
Local laptop (GPU)	NVIDIA GPU, 6 GB+ VRAM	Can run 7B models comfortably
Google Colab (Free)	Free T4 GPU, session limit ~12 hrs	Best free option for 7B models
Kaggle Notebooks	Free P100/T4 GPU, 30 hrs/week	Good backup if Colab quota is used up

2.2 Software Stack

- **Python 3.10+**
- **Ollama** – for running LLMs locally with a single command (<https://ollama.com>)
- **Hugging Face Transformers** – for models not available via Ollama, or for running on Colab GPUs
- **Evaluation libraries:** rouge-score, bert-score, evaluate, scikit-learn
- **Data handling:** pandas, datasets (Hugging Face)
- **Visualization:** matplotlib, seaborn
- **Optional dashboard:** streamlit or gradio

2.3 Installation Commands

```
# Create environment
python -m venv edubench-env
source edubench-env/bin/activate # Windows: edubench-env\Scripts\activate

# Install core libraries
pip install pandas numpy matplotlib seaborn
pip install rouge-score bert-score evaluate
pip install datasets transformers accelerate torch
pip install ollama # Python client for Ollama
pip install streamlit # optional, for dashboard

# Install Ollama (run in terminal, not pip)
# Mac/Linux:
curl -fsSL https://ollama.com/install.sh | sh
# Windows: download installer from https://ollama.com/download
```

2.4 Pulling the Local Models

```
ollama pull llama3.2:3b
ollama pull mistral:7b
ollama pull gemma2:2b
ollama pull phi3:mini
ollama pull qwen2.5:3b
```

Low-RAM Alternative

If the laptop has ≤ 8 GB RAM, replace the 7B model with a smaller variant, e.g. `qwen2.5:1.5b` or `tinyllama`, and reduce batch size / max tokens generated.

3 Dataset Selection

3.1 Recommended Datasets (Free, Hugging Face Hosted)

Dataset	Description	HF Identifier
SciQ	Science exam questions with ground-truth answers and supporting evidence	<code>allenai/sciq</code>
OpenBookQA	Elementary science reasoning, multiple-choice	<code>openbookqa</code>
ARC (Easy/Challenge)	Grade-school science questions	<code>ai2_arc</code>
RACE	English reading comprehension (middle/high school exams)	<code>race</code>
SQuAD v1.1	General reading comprehension QA	<code>squad</code>

3.2 Recommended Combination

Use **SciQ** (science) + **RACE** (language/reading comprehension) to test models across two different educational domains. This cross-domain comparison is a key novelty point for the paper.

3.3 Loading a Dataset

```
from datasets import load_dataset

sciq = load_dataset("allenai/sciq", split="test")
race = load_dataset("race", "middle", split="test")

# Take a manageable, fixed sample for the study
sciq_sample = sciq.shuffle(seed=42).select(range(150))
race_sample = race.shuffle(seed=42).select(range(150))
```

Sample Size Guidance

150–300 questions per subject is enough for statistically meaningful comparison while keeping local inference time manageable (a few hours across 5 models on CPU).

4 Models Under Comparison

Model	Size	Source	Notes
Llama 3.2	3B	Meta (via Ollama)	Strong general-purpose baseline
Mistral	7B	Mistral AI (via Ollama)	Larger, often stronger reasoning
Gemma 2	2B	Google (via Ollama)	Lightweight, fast
Phi-3 Mini	3.8B	Microsoft (via Ollama/HF)	Trained with strong emphasis on reasoning/education-style data
Qwen 2.5	3B	Alibaba (via Ollama)	Competitive multilingual/reasoning model

Selection Rationale

The five models span multiple organizations and parameter scales (2B–7B), enabling both a “best overall model” analysis and a “does model size correlate with educational QA quality” analysis — a strong secondary research question for the paper.

5 Methodology / Pipeline

5.1 Pipeline Diagram (Textual)

- Step 1: Dataset Preparation** – Load and clean SciQ + RACE samples; standardize into a single schema: {id, subject, question, context, reference_answer}.
- Step 2: Prompt Templating** – Design one fixed prompt template applied identically across all models (see Section 5.2) to ensure fairness.
- Step 3: Answer Generation** – Run each of the 5 models on every question; store outputs with metadata (model name, latency, token count).
- Step 4: Metric Computation** – Compute ROUGE-L, BERTScore, and LLM-judge score for every (question, model-answer) pair.
- Step 5: Aggregation** – Average scores per model overall and per subject; compute variance and rank.
- Step 6: Statistical Testing** – Run paired significance tests (e.g., Wilcoxon signed-rank test) between top models to confirm differences are not due to chance.
- Step 7: Error / Qualitative Analysis** – Manually inspect a subset of low-scoring answers; categorize failure modes (hallucination, incompleteness, off-topic).
- Step 8: Reporting** – Generate leaderboard tables, bar/radar charts, and write up findings.

5.2 Fixed Prompt Template

```
PROMPT_TEMPLATE = """You are a helpful and knowledgeable tutor.  
Answer the following question clearly and concisely, in 2-4 sentences.  
  
Question: {question}  
{context_block}  
  
Answer: """
```

Why Fix the Prompt?

Using an identical prompt template across all models isolates the comparison to model capability rather than prompt engineering skill – essential for a fair benchmark. (Prompt-sensitivity can be a separate optional experiment, see Section 9.)

6 Evaluation Metrics (The Three Metrics)

6.1 Metric 1 – Lexical Overlap: ROUGE-L

Measures the longest common subsequence overlap between the generated answer and the reference answer. Captures surface-level wording similarity.

```
from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(['rougeL'], use_stemmer=True)

def compute_rouge(reference, generated):
    scores = scorer.score(reference, generated)
    return scores['rougeL'].fmeasure
```

6.2 Metric 2 – Semantic Similarity: BERTScore

Uses contextual embeddings to measure meaning-level similarity, robust to paraphrasing (important since a correct answer may be worded very differently from the reference).

```
from bert_score import score as bert_score

def compute_bertscore(references, generated_list):
    P, R, F1 = bert_score(generated_list, references, lang="en", verbose=False)
    return F1.tolist() # F1 per example
```

6.3 Metric 3 – Correctness / Quality: Local LLM-as-Judge

Since no paid API (e.g., GPT-4) is used for judging, the **largest locally available model** (e.g., Mistral-7B) is used as an impartial grader, scoring each answer 1–5 for factual correctness and completeness against the reference.

```
JUDGE_MODEL = "mistral:7b"

JUDGE_PROMPT = """You are grading a student-facing AI tutor's answer.

Question: {question}
Reference (correct) answer: {reference}
AI-generated answer: {generated}

Rate the AI-generated answer from 1 to 5 for factual correctness and
completeness relative to the reference answer.
Respond with ONLY a single integer from 1 to 5."""

def llm_judge_score(question, reference, generated):
    prompt = JUDGE_PROMPT.format(question=question, reference=reference, generated=
        generated)
    response = ollama.chat(model=JUDGE_MODEL, messages=[{"role": "user", "content":
        prompt}])
    text = response['message']['content'].strip()
    try:
        return int(''.join(filter(str.isdigit, text))[:1])
    except:
        return None # flag for manual review
```

Validity Note for the Paper

Because the judge is itself one of the compared models’ “family,” explicitly exclude Mistral-7B’s own answers from LLM-judge scoring bias discussion, and validate the judge against a small human-annotated subset (Section 9) to report inter-rater agreement (e.g., Cohen’s kappa).

6.4 Metric Summary Table

Metric	Category	What It Captures	Library
ROUGE-L	Lexical	Word/phrase overlap with reference	<code>rouge-score</code>
BERTScore (F1)	Semantic	Meaning-level similarity despite paraphrasing	<code>bert-score</code>
LLM-Judge (1–5)	Correctness/Quality	Factual accuracy, completeness	Local model via <code>ollama</code>

7 Aggregation, Ranking and Statistical Analysis

7.1 Aggregation Code

```
import pandas as pd

df = pd.DataFrame(results) # contains rouge, bertscore, judge columns per row

leaderboard = df.groupby("model").agg(
    avg_rouge=("rouge_l", "mean"),
    avg_bertscore=("bertscore_f1", "mean"),
    avg_judge=("judge_score", "mean"),
    avg_latency=("latency_sec", "mean")
).reset_index()

# Normalize each metric to 0-1 and compute a combined score
for col in ["avg_rouge", "avg_bertscore", "avg_judge"]:
    leaderboard[col + "_norm"] = (leaderboard[col] - leaderboard[col].min()) / \
        (leaderboard[col].max() - leaderboard[col].min())

leaderboard["combined_score"] = leaderboard[
    ["avg_rouge_norm", "avg_bertscore_norm", "avg_judge_norm"]
].mean(axis=1)

leaderboard = leaderboard.sort_values("combined_score", ascending=False)
print(leaderboard)
```

7.2 Subject-wise Breakdown

```
subject_leaderboard = df.groupby(["subject", "model"]).agg(
    avg_rouge=("rouge_l", "mean"),
    avg_bertscore=("bertscore_f1", "mean"),
    avg_judge=("judge_score", "mean")
).reset_index()
```

7.3 Statistical Significance Testing

```
from scipy.stats import wilcoxon

model_a_scores = df[df.model == "mistral:7b"]["judge_score"]
model_b_scores = df[df.model == "phi3:mini"]["judge_score"]

stat, p_value = wilcoxon(model_a_scores, model_b_scores)
print(f"p-value: {p_value}") # p < 0.05 => statistically significant difference
```

7.4 Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns

# Overall leaderboard bar chart
sns.barplot(data=leaderboard, x="model", y="combined_score")
plt.title("Overall Model Ranking (Combined Score)")
plt.savefig("leaderboard.png", dpi=200, bbox_inches="tight")
```

```
# Radar chart comparing metrics per model (recommended for the paper)  
# Subject-wise heatmap  
pivot = subject_leaderboard.pivot(index="model", columns="subject", values="avg_judge"  
    )  
sns.heatmap(pivot, annot=True, cmap="YlGnBu")  
plt.title("Judge Score by Subject and Model")  
plt.savefig("subject_heatmap.png", dpi=200, bbox_inches="tight")
```

8 Week-by-Week Roadmap

Week	Milestone	Detailed Tasks
1	Environment Setup & Literature Review	Install Python, Ollama, pull all 5 models; read 5–8 related papers (MMLU, HELM, educational NLP benchmarks) to write the Related Work section.
2	Dataset Preparation	Load SciQ and RACE from Hugging Face; clean and standardize schema; select fixed sample (150–300 Qs per subject); create train/held-out split if needed for judge validation.
3	Answer Generation Pipeline	Implement and test the fixed prompt template; run all 5 models on the full sample; save raw answers with latency/token metadata; sanity-check a handful of outputs manually.
4	Metric Implementation	Implement ROUGE-L and BERTScore scoring; implement and calibrate the LLM-judge prompt; run all three metrics across the full result set; save scored dataset.
5	Human Validation & Aggregation	Manually score 50 random samples (1–5 scale) to validate the LLM-judge; compute agreement (Cohen’s kappa); build leaderboard tables (overall + subject-wise); run significance tests.
6	Analysis & Visualization	Generate bar charts, radar charts, subject heatmaps, cost/latency-vs-accuracy plots; perform qualitative error analysis (hallucination categories, verbosity bias).
7	(Optional) Dashboard	Build a Streamlit/Gradio leaderboard app so results are interactively explorable; polish figures for the paper.
8	Paper Writing & Submission Prep	Write all sections (see Section 10); proofread; format to target venue/conference template; prepare code + dataset release (GitHub repo) for reproducibility.

9 Optional Extensions (For Stronger Papers / Bonus Marks)

- Ext. 1: Prompt Sensitivity Study** – Re-run generation with 3 prompt styles (direct, chain-of-thought, few-shot) and measure which model is most prompt-robust.
- Ext. 2: Cost/Latency vs. Accuracy Trade-off** – Plot combined score against average generation latency to identify the best “efficiency frontier” model – highly practical for ed-tech deployment discussions.
- Ext. 3: Hallucination Rate Analysis** – Use the LLM-judge (or manual review) to explicitly flag and count factually unsupported statements per model.
- Ext. 4: Model-Size vs. Performance Correlation** – Since models range 2B–7B parameters, plot parameter count against combined score to test whether “bigger is better” holds for educational QA.
- Ext. 5: Cross-Domain Robustness** – Compare each model’s rank on SciQ (science) vs. RACE (reading comprehension) to test subject-generalization.

10 Recommended Paper Structure

1. **Abstract** – 150–200 words summarizing problem, method, key finding.
2. **Introduction** – Motivation (LLMs as free/local tutoring tools), gap in existing benchmarks, contributions.
3. **Related Work** – LLM benchmarks (MMLU, HELM, BIG-bench), educational NLP evaluation, LLM-as-judge literature.
4. **Dataset** – SciQ + RACE description, sampling strategy, statistics table.
5. **Methodology** – Models used, prompt template, generation pipeline, three metrics with justification.
6. **Experimental Setup** – Hardware, software versions, reproducibility details (seed values, sample sizes).
7. **Results** – Overall leaderboard table, subject-wise table, charts, significance test results.
8. **Discussion** – Trade-offs between models (accuracy vs. verbosity vs. latency), what the three metrics individually reveal that a single metric would miss.
9. **Limitations** – Sample size, judge-model bias, English-only, subject coverage.
10. **Conclusion & Future Work** – Summary, multilingual/larger-scale extension ideas.
11. **References**

11 Deliverables Checklist

- ☐ Cleaned, sampled dataset file (`dataset_sample.json`)
- ☐ Raw model answers file (`raw_answers.json`)
- ☐ Scored results file (`scored_results.csv`)
- ☐ Leaderboard table (overall + subject-wise)

- ☐ At least 3 visualizations (bar chart, heatmap, efficiency-frontier plot)
- ☐ Human-validation sample and agreement score
- ☐ GitHub repository with code, README, and reproduction instructions
- ☐ Final paper (PDF, formatted to target venue)
- ☐ (Optional) Interactive Streamlit/Gradio leaderboard demo

12 Reference Tools and Documentation Links

- Ollama: <https://ollama.com>
- Hugging Face Datasets: <https://huggingface.co/docs/datasets>
- Hugging Face Transformers: <https://huggingface.co/docs/transformers>
- ROUGE Score library: <https://github.com/google-research/google-research/tree/master/rouge>
- BERTScore: https://github.com/Tiiiger/bert_score
- SciQ dataset: <https://huggingface.co/datasets/allenai/sciq>
- RACE dataset: <https://huggingface.co/datasets/race>
- Google Colab: <https://colab.research.google.com>
- Kaggle Notebooks: <https://www.kaggle.com/code>